Mentimeter

Laboratorul 9

Factory, Strategy, Observer

2

Mentimeter

Instructions

Mentimeter

Design patterns

- modeleaza relatiile si interactiunile dintre clase si obiecte
- ~2000 de design patterns

1

Mentimeter

Tipuri de design patterns

- Creational patterns (cum se creeaza obiectele)
- Structural patterns (cum le legam -> relatii)
- Behavioural patterns (cum comunica intre ele)

Mentimeter

Design patterns discutate deja

Mentimeter

Singleton

- maxim o singura instanta a unei clase
- anti-pattern (toate clasele depind de aceeasi instanta)

Mentimeter

Visitor

- tipuri diferite de operatii pe tipuri diferite de obiecte
- nerecomandat cand adaugam multe tipuri noi de obiecte

Mentimeter

Factory

- genereaza instante de clase inrudite
- inrudite = implementeaza aceeasi interfata, mostenesc aceeasi clasa abstracta
- vrem sa ne facem obiecte fara sa stim exact subtipul lor
- ascunde logica instantierii de subtipuri

1

Mentimeter

Factory patterns

- Simple factory pattern
- Factory Method pattern
- Abstract Factory pattern
- mai multe detalii https://refactoring.guru/design-patterns/factory-comparison

Mentimeter

Ierarhia de clase

Mentimeter

```
enum Right {
    NONE,
    SOME,
    ALL_IN
}

interface User {
    Right getRights();
}

class Student implements User {
    @Override
    public Right getRights() {
        return Right.NONE;
    }
}
```

Ierarhie 1

Mentimeter

```
class Teacher implements User {
    @Override
    public Right getRights() {
        return Right.SOME;
    }
}

class Admin implements User {
    @Override
    public Right getRights() {
        return Right.ALL_IN;
    }
}
```

Ierarhie 2

Mentimeter

Simple factory pattern

Mentimeter

```java
class UserFactory {
    public static User createUser(String type) {
        return switch (type) {
            case "student" -> new Student();
            case "teacher" -> new Teacher();
            case "admin" -> new Admin();
            default -> null;
        };
    }
}

public class Main {
    public static void main(String[] args){
        User student = UserFactory.createUser( type: "student");
        System.out.println(student.getRights());
    }
}
```

Simple factory



Mentimeter

Factory method pattern

→ cate un factory pentru fiecare subtip

Mentimeter

```java
interface MethodUserFactory {
  User createUser(String type);
}

class StudentFactory implements MethodUserFactory {
  @Override
  public User createUser(String type) {
    return new Student();
  }
}

class TeacherFactory implements MethodUserFactory {
  @Override
  public User createUser(String type) {
    return new Teacher();
  }
}
```

Method factory 1

Mentimeter

```
class AdminFactory implements MethodUserFactory {
  @Override
  public User createUser(String type) {
    return new Admin();
  }
}

public class FactoryMethodMain {
  public static void main(String[] args) {
    StudentFactory studentFactory = new StudentFactory();
    User student = studentFactory.createUser( type: "student");

    System.out.println(student.getRights());
  }
}
```

Method factory 2

Mentimeter

Abstract factory pattern

→ cate un factory pentru o ierarhie de obiecte

Mentimeter

```java
interface Assignment {
    void edit();
    void grade();
    void view();
}

class EditableAssignment implements Assignment {
    @Override
    public void edit() {
        System.out.println("Editing...");
    }

    @Override
    public void grade() { System.out.println("Grading..."); }

    @Override
    public void view() { System.out.println("Viewing..."); }
}
```

Assignment cu drepturi diferite 1

Mentimeter

```java
class GradableAssignment implements Assignment {
    @Override
    public void edit() {
        System.out.println("You can't edit this");
    }

    @Override
    public void grade() { System.out.println("Grading..."); }

    @Override
    public void view() { System.out.println("Viewing..."); }
}

class ViewOnlyAssignment implements Assignment {
    @Override
    public void edit() {
        System.out.println("You can't edit this");
    }

    @Override
    public void grade() { System.out.println("You can't grade this"); }

    @Override
    public void view() { System.out.println("Viewing..."); }
}
```

Assignment cu drepturi diferite 2

Mentimeter

Cate un factory pentru fiecare ierarhie de obiecte

- 2 ierarhii diferite: user (admin/teacher/student), assignment (view-only/gradable/editable)
- pentru fiecare tip de acces avem un user/assignment diferit
- vom face cate un factory pentru fiecare tip de acces

Mentimeter

```
public interface RightsAbstractFactory {
    User createUser();
    Assignment createAssignment();
}
```

Interfata Abstract Factory

Mentimeter

```
public class NoRightsAbstractFactory implements RightsAbstractFactory {
    @Override
    public User createUser() { return new Student(); }

    @Override
    public Assignment createAssignment() {
        return new ViewOnlyAssignment();
    }
}
```

Mentimeter

```
public class SomeRightsAbstractFactory implements RightsAbstractFactory {
    @Override
    public User createUser() { return new Teacher(); }

    @Override
    public Assignment createAssignment() { return new GradableAssignment(); }
}
```

Mentimeter

```java
public class AllRightsAbstractFactory implements RightsAbstractFactory {
    @Override
    public User createUser() { return new Admin(); }

    @Override
    public Assignment createAssignment() { return new EditableAssignment(); }
}
```

Mentimeter

Singleton Factory

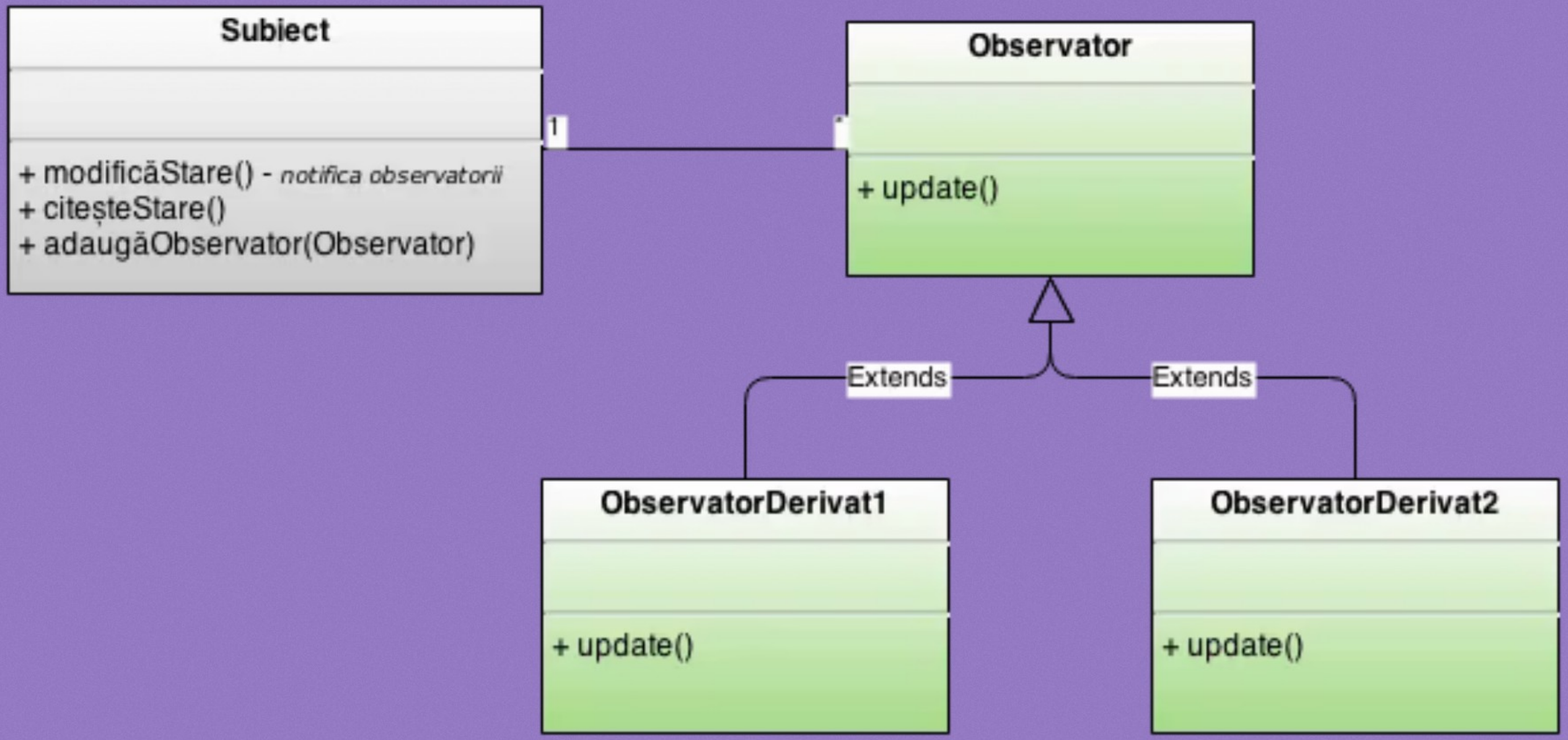
→ avem nevoie de o singura instanta a clasei factory

Mentimeter

Observer Pattern

- dependenta 1 la n intre obiecte
- un obiect isi schimba starea => toti dependentii sunt notificati si actualizati automat
- design pattern behavioral (organizeaza comunicarii dintre clase in functie de rolurile lor)

Mentimeter



Structura

Mentimeter

Structura

- 2 tipuri de obiecte: Subiect, Observator
- subiectul are date care sunt folosite de observatori
- fiecare subiect isi stie toti observatorii si iti poate adauga unii noi
- observatorii sunt notificati automat de schimbari

Mentimeter

Subiect

- mentine o lista de referinte cu observatori
- ofera metode de abonare/dezabonare a observatorilor
- NU trebuie sa stie ce face un observator cu datele, e suficient numai sa il notifice

Mentimeter

Observator

- interfata Obsservator
- expune metode care vor fi apelate de catre Subiect la schimbarea starii
- toti observatorii unui anumit tip trebuie sa implementeze aceasta interfata

Mentimeter

View/ObservatorDerivat

→ implementeaza interfata Observator

Mentimeter

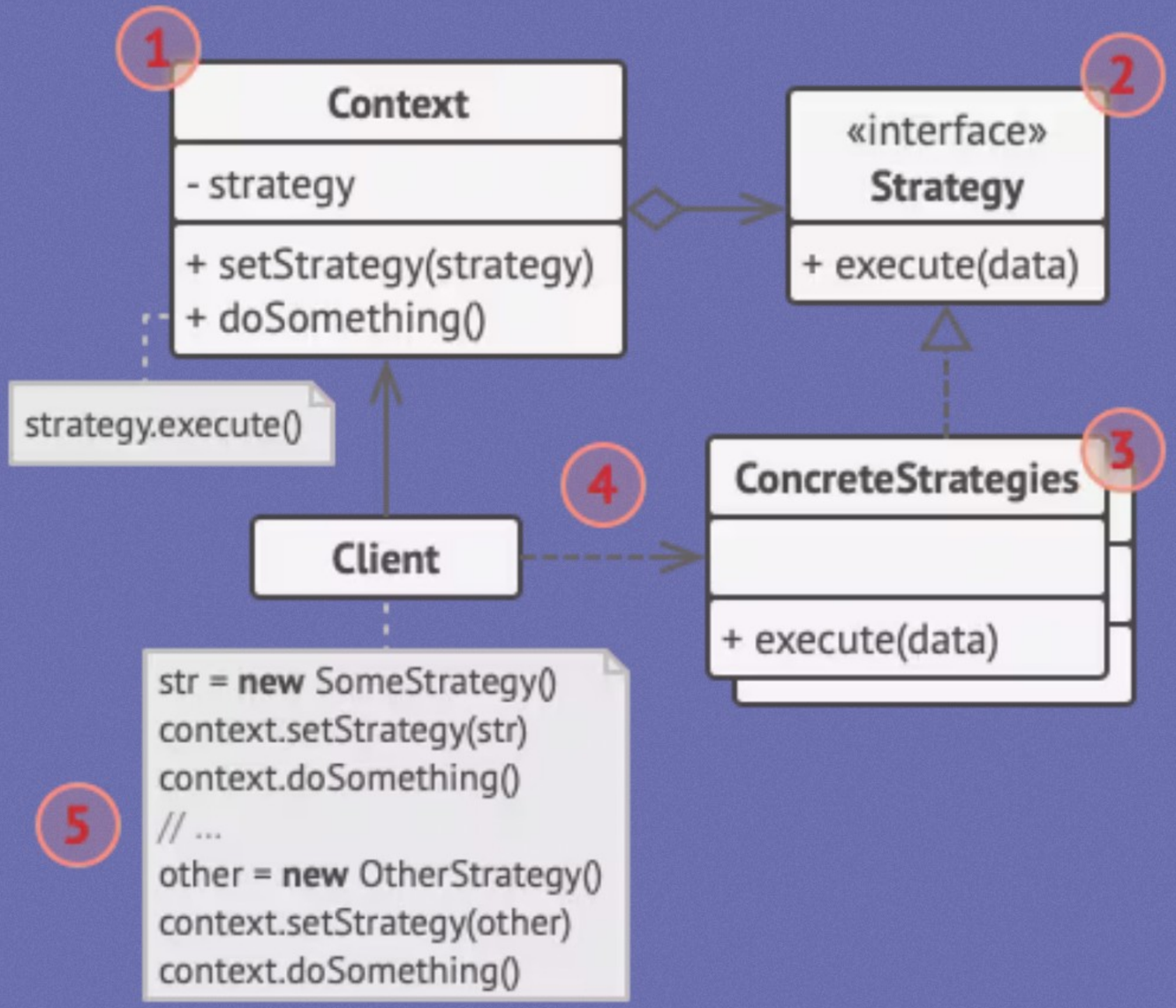
Demo

Mentimeter

Strategy Pattern

- incapsulam strategiile in clasa
- selectam strategia la runtime
- behavioral pattern (decuplarea strategiilor și folosirea lor independentă în funcție de situația de la runtime)

Mentimeter



Structura

1
?

Mentimeter

Strategy

- interfata comuna a strategiilor
- are metoda execute care aplica strategia pe niste date

Mentimeter

ConcreteStrategy

- implementeaza interfata Strategy
- ofera o implementare a metodei execute

Mentimeter

Context

- depinde de strategie
- are metode care executa strategia pe anumite date

Mentimeter

Client

- are un context (ex: date) care depind de o strategie
- apeleaza metodele contextului cu strategii diferite

Mentimeter

Cand folosim Strategy?

- mai multe implementari (strategii) pentru un algoritm
- nu vrem sa depindem de o anumita implementare (cuplare nu este prea stransa)
- vrem sa alegem dinamic implementarea

Mentimeter

Demo

Mentimeter

Principii de design adresate de aceste patternuri

- Factory -> Dependency injection principle
- Strategy -> Separarea codului care se schimbă de cel care rămâne la fel
- Observer -> Loosly coupled design

Mentimeter

Dependency Injection principle

- nu vrem sa depindem de implementari specifice (subtipuri)
- ne definim tipuri abstracte (interfete/clase abstracte)
- folosim tipurile abstracte, nu subtipurile

Mentimeter

Separarea codului care se schimbă de cel care rămâne la fel

- nu folosim Strategy -> daca modificam algoritmul, trebuie actualizat peste tot unde l-am inclus
- nu ar trebui sa ne pese de implementare, ci numai de outcome (semnaturile functiilor)
- folosim un tip abstract pentru strategie ca sa nu mai depindem de o anumita implementare

Mentimeter

Loosly coupled design

- vrem sa facem cat mai putine schimbari (sau nada) atunci cand adaugam ceva nou
- Observer -> pot oricand sa adaug un nou observator, nu depind de subtipul lui